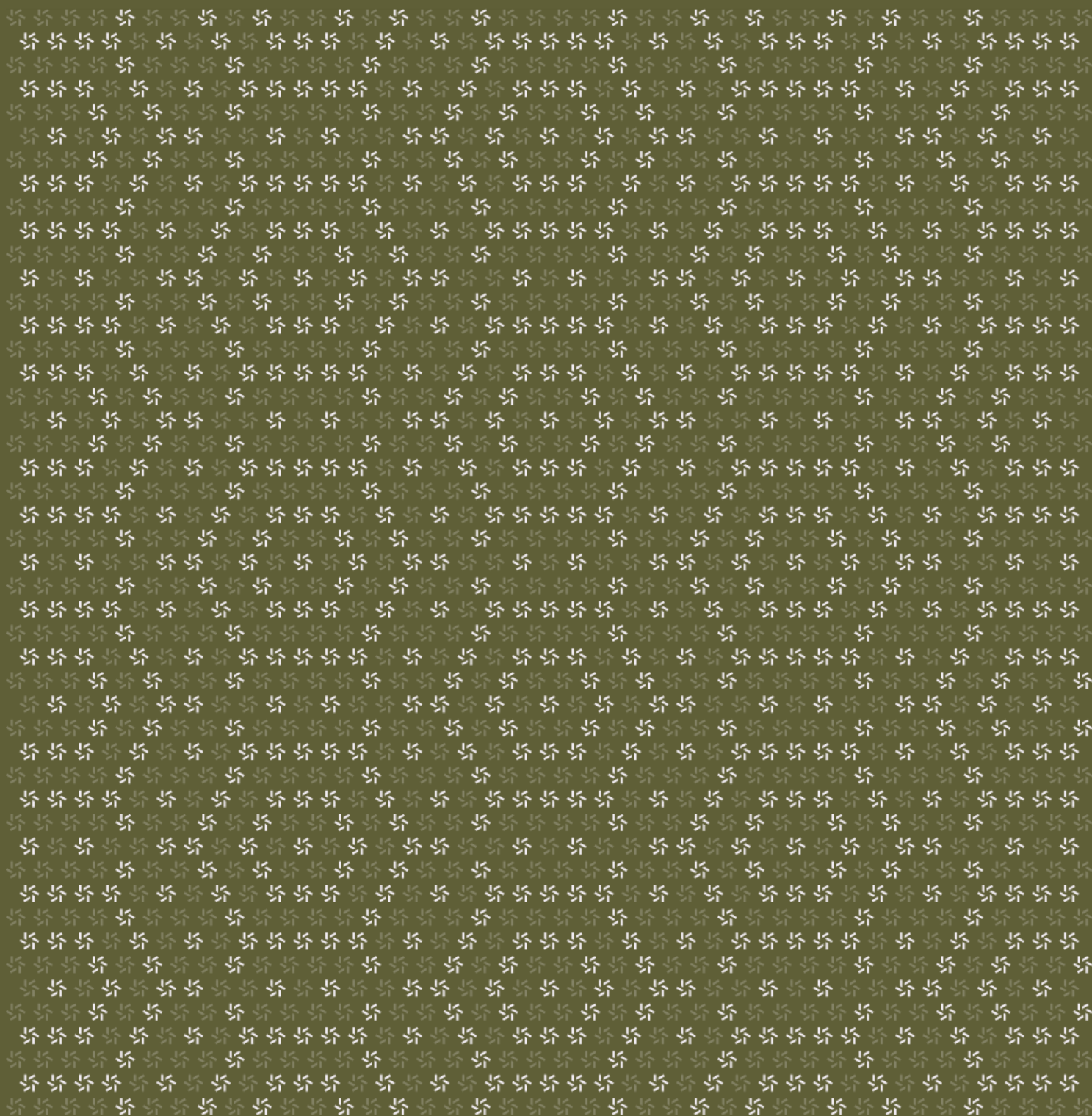


March 25, 2026

Celestia op-alt-da

Application Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	6
2. Introduction	6
2.1. About Celestia op-alt-da	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Unnecessary postsubmit read in compact mode can falsely fail successful blob submissions	11
3.2. Noncanonical commitment allows unbounded S3 object creation via GET	14
3.3. Fallback reads serve data without integrity verification against the requested commitment	17
3.4. Celestia terminal errors possibly masked by fallback read	19
4. Discussion	20
4.1. Configured transaction priority is never applied	21

5.	System Design	22
5.1.	Component: DA server	23
5.2.	Component: Celestia storage	24
5.3.	Component: Fallback storage	26

6.	Assessment Results	27
6.1.	Disclaimer	28

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Celestia Labs from March 20th to March 25th, 2026. During this engagement, Zellic reviewed Celestia op-alt-da's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Could the DA server return data that does not match the requested commitment?
 - Could the fallback storage path be abused to serve tampered or inconsistent data?
 - Are there input-validation weaknesses in the HTTP or commitment-parsing layers that could lead to unexpected behavior?
 - Could sensitive information be exposed through error messages, logs, or process metadata?
 - Are configuration options applied correctly and behave as documented?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- All autogenerated files, tests, and mocks
- The Celestia node software, consensus layer, or bridge-node implementations
- The OP Stack components that consume the DA server (e.g., op-batcher, op-node)
- Third-party dependencies and libraries
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Celestia op-alt-da files, we discovered four findings, all of which were low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Celestia Labs in the Discussion section ([4.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
 Critical	0
 High	0
 Medium	0
 Low	4
 Informational	0



2. Introduction

2.1. About Celestia op-alt-da

Celestia Labs contributed the following description of Celestia op-alt-da:

A Celestia DA server for OP Stack's Alt-DA mode using generic commitments. The server provides a simple HTTP API that the OP Stack's op-batcher uses to store and retrieve data availability commitments on Celestia.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Unsafe code patterns. Many vulnerabilities in web-based applications arise from oversights during development. Missing an authentication check on a single API endpoint can critically impact the security of the overall application. Failure to properly sanitize and encode user input can lead to vulnerabilities like SQL injection, server-side request forgery, and more. We use both automated tools and extensive manual review to identify unsafe code patterns.

Business logic errors. Business logic is the heart of all web applications. We carefully review logic to ensure that the code securely and correctly implements the specified functionality. We also thoroughly examine all specifications for inconsistencies, flaws, and vulnerabilities, including for risks of fraud and abuse.

Integration risks. Web projects frequently have many third-party dependencies and interact with services and libraries that are not under the developer's control. We review the project's external interactions and identify potentially dangerous behavior resulting from compatibility issues and data inconsistencies.

Code maturity. We look for possible improvements across the entire codebase, reviewing violations of industry best practices and code quality standards. We suggest improvements to code clarity, documentation, and testing practices.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather

than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped files itself. These observations — found in the Discussion (4.7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

DRAFT

2.3. Scope

The engagement involved a review of the following targets:

Celestia op-alt-da Files

Type	Go
Platform	OP Stack (Optimism Alt-DA)
Target	op-alt-da
Repository	https://github.com/celestiaorg/op-alt-da
Version	0b3c3e5c7b05a83be759072316cd5e51303d103a
Programs	*.go

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of eight person-days. The assessment was conducted by two consultants over the course of four calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Leonardo Veronese
✈ Engineer
leonardo@zellic.io ↗

Seokchan Yoon
✈ Engineer
seokchan@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 20, 2026 Start of primary review period

March 23, 2026 Kick-off call

March 25, 2026 End of primary review period

3. Detailed Findings

3.1. Unnecessary postsubmit read in compact mode can falsely fail successful blob submissions

Target	celestia_storage.go		
Category	Business Logic	Severity	Low
Likelihood	Medium	Impact	Low

Description

Celestia's node architecture separates the write and read paths into independent infrastructure. Blob submissions (writes) go through a consensus full node via CoreGRPC, while blob lookups (reads) go through a bridge node via JSON-RPC. These are independent systems, and a block confirmed by consensus may not be immediately available on the bridge node due to propagation delay.

In `submitBlob()`, after `Client.Submit()` successfully confirms a blob on the consensus node, compact mode performs an additional `Client.Get()` call to fetch share metadata from the bridge node:

```
// celestia_storage.go
func (d *CelestiaStore) submitBlob(ctx context.Context, data []byte) ([]byte,
[]byte, error) {
    // [...]
    height, err := d.Client.Submit(ctx, []*blob.Blob{b}, state.NewTxConfig())
    if err != nil {
        return nil, nil, err
    }
    // Submit succeeded, blob is now confirmed on Celestia

    blobID := CelestiaBlobID{
        Height:    height,
        Commitment: b.Commitment,
        isCompact: d.CompactBlobID,
    }

    if d.CompactBlobID {
        b, err = d.Client.Get(ctx, height, d.Namespace, b.Commitment)
        if err != nil {
            return nil, nil, err // entire PUT fails despite successful
            submission
        }
        blobID.ShareOffset = uint32(b.Index())
    }
}
```

```
        blobID.ShareSize = uint32(size)
    }
    // [...]
}
```

However, compact mode's `MarshalBinary()` only serializes `Height` and `Commitment`, discarding the `ShareOffset` and `ShareSize` fields entirely:

```
// celestia_storage.go
func (c *CelestiaBlobID) MarshalBinary() ([]byte, error) {
    // [...]
    if c.isCompact {
        id := make([]byte, CompactBlobIDSize) // 40 bytes
        binary.LittleEndian.PutUint64(id[0:HeightSize], c.Height)
        copy(id[HeightSize:HeightSize+CommitmentSize], c.Commitment)
        return id, nil // ShareOffset and ShareSize are not included
    }
    // [...]
}
```

The `Get()` call in compact mode is therefore unnecessary. It fetches data that is computed and then discarded. The most realistic trigger is a read-your-own-write race condition: `Submit()` confirms a block on the consensus node, then `Get()` immediately tries to read that same block from the bridge node before it has synced. This can occur during normal operation as a natural consequence of the write/read infrastructure split.

Both operations also share the same request context, so if `Submit()` consumes most of the time-out budget, `Get()` can fail with a deadline-exceeded error.

Impact

When the `Get()` call fails, the entire PUT request returns an HTTP 500 error even though the blob was already successfully submitted to Celestia. The OP Stack's `op-batcher` retries on HTTP 500, submitting the same blob again and paying TIA gas fees a second time.

Recommendations

Skip the `Client.Get()` call when compact mode is enabled, as the share metadata it fetches is not used in compact blob ID serialization. If share metadata is needed for other purposes, consider fetching it only in full-ID mode or adding a retry mechanism with backoff to account for bridge-node propagation delay.

Remediation

TBD

DRAFT

3.2. Noncanonical commitment allows unbounded S3 object creation via GET

Target	celestia_storage.go		
Category	Coding Mistakes	Severity	Low
Likelihood	Low	Impact	Low

Description

The server accepts commitment bytes of any length beyond the minimum 40 bytes without enforcing a canonical length. This creates a mismatch between how commitments are consumed by Celestia (only the leading bytes) and how they are used as S3 fallback keys (full input including trailing bytes).

The `parseCommitment()` function decodes the hex from the URL path with no length validation:

```
// celestia_server.go
func (d *CelestiaServer) parseCommitment(urlPath string) ([]byte, error) {
    key := path.Base(urlPath)
    comm, err := hexutil.Decode(key)
    // [...]
    return comm, nil
}
```

The `UnmarshalBinary()` function only requires `>= 40` bytes and silently ignores anything beyond that:

```
// celestia_storage.go
func (c *CelestiaBlobID) UnmarshalBinary(data []byte) error {
    if len(data) < CompactBlobIDSize {
        return fmt.Errorf("invalid ID length: expected at least %d bytes (compact format), got %d", CompactBlobIDSize, len(data))
    }
    c.Height = binary.LittleEndian.Uint64(data[0:HeightSize])
    c.Commitment = make([]byte, CommitmentSize)
    copy(c.Commitment, data[HeightSize:HeightSize+CommitmentSize])
    // data[40:] silently ignored
    // [...]
}
```

The Celestia lookup succeeds using just the leading bytes. On success, the server passes the full original commitment (including trailing junk) to the fallback write:

```
// celestia_server.go
func (d *CelestiaServer) getBlob(ctx context.Context, comm []byte) ([]byte,
error) {
    // [...]
    data, celestiaErr := d.store.Get(getCtx, comm)
    if celestiaErr == nil {
        if d.fallback.Available() {
            d.wg.Add(1)
            go d.putFallback(context.Background(), comm, data) // full comm
        }
        return data, nil
    }
    // [...]
}
```

The S3 provider generates keys from keccak256(commitment), so different trailing bytes produce different S3 keys for the same blob data:

```
// fallback/s3/s3.go
func (p *S3Provider) makeKey(commitment []byte) string {
    keccakKey := crypto.Keccak256(commitment)
    return path.Join(p.prefix, hex.EncodeToString(keccakKey))
}
```

Given a valid commitment C, an attacker can send GET /get/0x<C>aa, GET /get/0x<C>bb, GET /get/0x<C>cc, and so on. Each request succeeds from Celestia (same blob) but creates a new S3 object (different Keccak hash). Each GET request is a few bytes but triggers a PUT of the full blob data (up to 2 MB). No authentication is required; only knowledge of one valid commitment, which is public information posted on L1.

Note that the server binds to 127.0.0.1 by default and was confirmed as internal-only during the project kickoff. An external attacker cannot reach the endpoint in the default deployment, which limits the practical exploitability of this issue.

Impact

An internal caller with access to the DA server can cause unbounded S3 storage consumption and S3 PUT cost amplification through GET-only requests. Each request also spawns a new goroutine for the fallback write. However, since the server is not externally reachable in the default configuration, the impact is limited to S3 cost amplification with no effect on data integrity or TIA spending.

Recommendations

Enforce canonical commitment length by rejecting GET requests where the decoded commitment does not match the expected length (42 bytes for compact and 50 bytes for full format, including the two-byte version prefix).

Alternatively, reserialize the parsed BlobID to canonical form before passing it to the fallback provider so that trailing bytes are stripped before the S3 key is derived.

Remediation

TBD

3.3. Fallback reads serve data without integrity verification against the requested commitment

Target	celestia_server.go		
Category	Protocol Risks	Severity	Medium
Likelihood	Low	Impact	Low

Description

When `getBlob()` falls back to S3 after a Celestia failure, the data returned from S3 is served directly to the client with no verification that it matches the requested commitment.

```
// celestia_server.go
func (d *CelestiaServer) getBlob(ctx context.Context, comm []byte) ([]byte, error) {
    // [...]

    data, celestiaErr := d.store.Get(getCtx, comm)
    if celestiaErr == nil {
        // [...]
        return data, nil
    }

    // Celestia failed, try fallback
    data, err := d.getFallback(ctx, comm)
    if err != nil {
        // [...]
    }

    // no verification that data matches comm
    return data, nil
}
```

The S3 key is derived from `keccak256(commitment)`, not from the data content itself:

```
// fallback/s3/s3.go
func (p *S3Provider) makeKey(commitment []byte) string {
    keccakKey := crypto.Keccak256(commitment)
    return path.Join(p.prefix, hex.EncodeToString(keccakKey))
}
```

The key is a hash of the *pointer* (the commitment), not a hash of the *content* (the blob data). There is nothing tying the stored data back to the commitment. If the S3 bucket is compromised (e.g., IAM misconfiguration, credential leak, insider threat), an attacker can replace any object with arbitrary data and the server will serve it as the original blob.

This matters because the fallback path triggers on any error from `store.Get()`, not only when Celestia is fully unavailable. Transient errors, time-outs, and even terminal errors such as `ErrNotFound` all cause the server to consult the fallback store. When the fallback succeeds, the server becomes the sole source of truth for blob data with no way to cross-check against Celestia.

However, this issue requires S3 bucket compromise, a Celestia error on the read path, and fallback being enabled, all at the same time. If S3 is already compromised, that is a severe infrastructure incident on its own. The integrity verification is best understood as a defense-in-depth measure rather than a mitigation for an independent vulnerability.

Impact

Under the triple precondition (S3 compromised, Celestia error on read path, fallback enabled), the server will serve arbitrary data as if it were the requested blob. For a DA server backing a rollup, this could mean a rollup verifier receives incorrect transaction data, or fault proofs and validity proofs operate on tampered data. The integrity guarantee that Celestia provides is bypassed entirely through the fallback path.

Recommendations

After reading from S3 fallback, verify the data against the requested commitment before returning it. Concretely, parse the requested commitment, reconstruct the Celestia blob from the fallback data under the configured namespace, and compare the derived blob commitment to the commitment embedded in the request.

Remediation

TBD

3.4. Celestia terminal errors possibly masked by fallback read

Target	celestia_server.go		
Category	Integration Risks	Severity	Low
Likelihood	Low	Impact	Low

Description

When `getBlob()` falls back to S3 after a Celestia failure, no distinction is made between transient errors (time-outs, connection failures) and terminal ones (`ErrNotFound`). The `isNotFoundError` check only runs if the fallback also fails; if S3 has the data, the Celestia error is silently dropped and the server returns successfully.

```
// celestia_server.go:273-311
func (d *CelestiaServer) getBlob(ctx context.Context, comm []byte) ([]byte, error) {
    data, celestiaErr := d.store.Get(getCtx, comm)
    if celestiaErr == nil {
        // success, read-through cache to S3
        go d.putFallback(context.Background(), comm, data)
        return data, nil
    }

    // Celestia failed. try fallback if available
    // no distinction between transient errors and terminal errors like
    // NotFound
    data, err := d.getFallback(ctx, comm)
    if err != nil {
        // only if fallback ALSO fails, the original celestia error is returned
        if isNotFoundError(celestiaErr) {
            return nil, altda.ErrNotFound
        }
        return nil, celestiaErr
    }

    // fallback succeeded, return data, celestiaErr is silently discarded
    return data, nil
}
```

Since the server populates the S3 fallback on every successful PUT and GET request, a blob that Celestia later reports as unavailable will have a copy in S3.

```
// celestia_server.go:280-282 (on every successful GET, read-through caching)
if d.fallback.Available() {
    go d.putFallback(context.Background(), comm, data)
}

// celestia_server.go:420-423 (on every successful PUT)
if d.fallback.Available() {
    go d.putFallback(context.Background(), commitment, input)
}
```

The practical severity depends on how often Celestia can return `ErrNotFound` for a previously submitted blob (e.g., due to pruning). Even if such scenario is unlikely, classifying errors before consulting the fallback is still good defensive practice.

Impact

A Celestia `ErrNotFound` is a terminal error that signals that a blob is not available on the DA layer. If masked by the S3 fallback, the rollup continues progressing as if the blob was present on Celestia when in reality it is not.

Recommendations

Only fall back to S3 for transient errors such as time-outs or connection failures. Terminal errors, including `ErrNotFound` and invalid commitment errors, should be returned directly to the caller without consulting S3.

Remediation

TBD

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Configured transaction priority is never applied

The server exposes `submission.tx_priority` as a configuration option for controlling transaction fee bidding and inclusion speed:

```
[submission]
tx_priority = 2 # 1=low, 2=medium, 3=high
```

The value is parsed into the configuration struct and validated to be within the range 1 to 3:

```
// cmd/daserver/config.go
type SubmissionConfig struct {
    Timeout      string `toml:"timeout"`
    TxPriority    int    `toml:"tx_priority" // 1=low, 2=medium, 3=high
    MaxBlobSize  string `toml:"max_blob_size"`
}

// [...]

if c.Submission.TxPriority < 1 || c.Submission.TxPriority > 3 {
    return fmt.Errorf("submission.tx_priority must be 1 (low), 2 (medium), or 3 (high)")
}
```

However, the actual blob submission always constructs a default `TxConfig` with no options, ignoring the configured priority entirely:

```
// celestia_storage.go
height, err := d.Client.Submit(ctx, []*blob.Blob{b}, state.NewTxConfig())
```

There is no code path that passes the configured value into `state.WithTxPriority()` or any equivalent option. An operator setting `tx_priority = 3` to improve inclusion speed under congestion, or `tx_priority = 1` to reduce gas costs, will see no effect. Because the setting is validated and documented, this mismatch is difficult to notice during operations.

To resolve this, the configured priority should be wired through to the `Submit()` call (e.g., `state.NewTxConfig(state.WithTxPriority(cfg.TxPriority))`).

Alternatively, if priority control is intentionally unsupported, the setting should be removed from the configuration surface and documentation to avoid misleading operators.

DRAFT

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: DA server

Description

The `CelestiaServer` is the HTTP server that exposes the DA interface to the OP Stack's `op-batcher`. It listens on a configurable address (default `127.0.0.1:3100`) and provides three endpoints:

1. `PUT/POST /put` — accepts raw blob data, submits it to Celestia via the storage layer, and returns raw generic-commitment bytes
2. `GET /get/<commitment>` — parses the commitment from the URL path, retrieves the blob from Celestia (or fallback if enabled), and returns the raw data
3. `GET /health` — returns HTTP 200

On `PUT / POST`, the server reads the request body (capped by `maxBlobSize`) and delegates to `store.Put()`. The storage layer returns an already encoded `alt-da.GenericCommitment`, which the server writes back to the client. On `GET`, the server calls `getBlob()`, which first tries Celestia and, if fallback is available, populates the fallback on success (read-through) or reads from fallback on Celestia failure.

The server applies `securityHeadersMiddleware` (`X-Content-Type-Options`, `X-Frame-Options`) and `recoveryMiddleware` (panic recovery) to all routes. An optional Prometheus metrics server runs on a separate port.

Invariants

- A commitment must not be returned by `PUT` unless `Client.Submit()` has confirmed the blob on Celestia.
- Though `GET` is intended to return data that corresponds to the requested commitment, the fallback path at the time of writing does not reverify this property against the commitment (see Finding [3.3](#) ↗).
- Request body size is bounded by `maxBlobSize` to prevent resource exhaustion on the write path.

Test coverage

Cases covered

- The `HandleHealth` returning HTTP 200
- The `HandlePut` with an empty body (400), incorrect route (400), successful submission (200), and internal error (500)
- The `HandleGet` with invalid hex (400), incorrect route (400), too-short commitment, successful retrieval (200), not found (404), and internal error (500)
- Prometheus metrics recording and registration
- Endpoint routing table verification

Cases not covered

- A live PUT-to-GET round trip only covered by a skipped integration test behind the integration build tag (`tests/integration/put_get_test.go`), not by the default unit-test path
- Concurrent request handling under load
- Fallback read-through and write-through behavior with a configured fallback provider
- The `HandlePut` with oversized body exceeding `maxBlobSize`

Attack surface

- The `parseCommitment()` function accepts hex-encoded commitments of arbitrary length. Trailing bytes beyond the expected blob ID size are silently accepted and propagated to the fallback layer (see Finding [3.2](#), ↗).
- The fallback read path returns S3 data without verifying it against the requested commitment (see Finding [3.3](#), ↗).
- Error messages from `Client.Submit()` are returned to the HTTP client and may include internal infrastructure details such as RPC endpoints and IP addresses.
- The server binds to `127.0.0.1` by default. If reconfigured to bind to `0.0.0.0`, all endpoints become externally reachable with no authentication.

5.2. Component: Celestia storage

Description

The `CelestiaStore` handles blob submission to and retrieval from the Celestia network. It manages two distinct infrastructure paths:

1. **Write path** — `submitBlob()` creates a `blob.Blob` under the configured namespace, submits it to a consensus full node via `Client.Submit()` (CoreRPC), and returns a marshaled `CelestiaBlobID`.

2. **Read path** — `Get()` parses a `CelestiaBlobID` from the commitment bytes then fetches the blob from a bridge node via `Client.Get()` (JSON-RPC).

The `CelestiaBlobID` supports two serialization formats:

1. **Compact** (40 bytes) — Height (eight bytes) plus Commitment (32 bytes) — this is the default
2. **Full** (48 bytes) — compact fields plus `ShareOffset` (four bytes) + `ShareSize` (four bytes)

When `TxClientConfig` is provided, the store also initializes the Celestia key ring (local file system or AWS KMS) for transaction signing and establishes the RPC-/CoreGRPC-backed client on startup.

Invariants

- A blob ID returned by `submitBlob()` must contain the correct Height and Commitment from a confirmed submission.
- The `UnmarshalBinary()` must correctly parse both compact and full-format blob IDs.
- Full-format IDs round trip all serialized fields through `MarshalBinary()` and `UnmarshalBinary()`.
- Compact-format IDs only preserve Height and Commitment; `ShareOffset` and `ShareSize` are dropped during compact serialization.

Test coverage

Cases covered

- Functions `MarshalBinary` / `UnmarshalBinary` round trip for compact, full, and zero-value formats.
- Moreover, `Validate()` rejects nil commitment and incorrect-size commitment.
- The `MarshalBinary` function fails on invalid fields (nil commitment, incorrect size).
- The `UnmarshalBinary` function rejects inputs that are too short, very short, or empty.
- Field-size constants match expected values.

Cases not covered

- Behavior of `UnmarshalBinary` with inputs longer than expected (trailing bytes are silently ignored — see Finding 3.2, ↗)
- The interaction between `Submit()` and `Get()` in compact mode under propagation delay (see Finding 3.1, ↗)
- Key-ring initialization failure paths for both local and AWS KMS backends

Attack surface

- The `UnmarshalBinary()` function accepts any input ≥ 40 bytes and silently ignores trailing data. This allows noncanonical representations of the same blob ID, which propagate to the fallback layer as distinct keys.
- In compact mode, `submitBlob()` performs an unnecessary `Client.Get()` after a successful `Submit()`. The fetched share metadata is discarded during compact serialization, but a failure in `Get()` causes the entire PUT to fail despite the blob already being confirmed on Celestia.
- The `TxConfig` passed to `Client.Submit()` is always default-constructed. The configured `tx_priority` value is validated but never wired through (see section [4.1](#), ↗).

5.3. Component: Fallback storage

Description

The fallback storage layer provides an optional S3-backed redundancy layer for blob data. It is used in three scenarios:

1. **Write-through.** After a successful Celestia submission, the blob is asynchronously written to S3 in a background goroutine via `putFallback()`.
2. **Read-fallback.** If a Celestia `Get()` fails (time-out, not found, or any other error), the server attempts to retrieve the blob from S3 via `getFallback()`.
3. **Read-through.** If a Celestia `Get()` succeeds, the blob is also written to S3 for future requests.

The `fallback.Provider` interface defines `Put()`, `Get()`, `Available()`, `Name()`, and `Timeout()`. The `S3Provider` implementation generates S3 object keys as `prefix/hex(keccak256(commitment))` and supports static credentials, environment credentials, and IAM role authentication.

When fallback is disabled, a `NoopProvider` is used. Its `Available()` method returns `false`, so all fallback code paths are skipped.

Invariants

- Fallback writes must be nonblocking; failures are logged as warnings but do not affect the HTTP response to the client.
- Fallback reads are only attempted after the primary Celestia read path returns an error; they are not limited to pure availability failures.
- S3 keys must be deterministic; the same commitment must always map to the same S3 object key.

Test coverage

Cases covered

- The `NewS3Provider` rejects missing bucket, invalid credential type, and static credentials without keys.
- The `makeKey()` function produces deterministic keys and correctly prepends the prefix.
- When the client is nil or the bucket is empty, `Available()` returns `false`.
- Both `Name()` and `Timeout()` return expected values.

Cases not covered

- Behavior of `makeKey()` with noncanonical (extra-length) commitments producing different keys for the same logical blob (see Finding [3.2](#) ↗)
- Direct assertions that omitted region and time-out being defaulted to `us-east-1` and `30s`, respectively
- Data-integrity verification on fallback reads (see Finding [3.3](#) ↗)
- Actual S3 Put and Get operations (no integration test with real or mocked S3)
- Concurrent write-through goroutine accumulation under sustained load

Attack surface

- S3 object keys are derived from `keccak256(commitment)`, where `commitment` is the raw input bytes including any trailing data. Noncanonical commitments produce distinct S3 keys for the same blob, enabling unbounded storage amplification.
- Data retrieved from S3 is returned to the client without verifying it against the requested commitment. If the S3 bucket is compromised, tampered data is served as if it were the original blob.
- Each successful Celestia GET and each fallback write spawns a new background goroutine. There is no concurrency limit or deduplication, so a burst of requests for the same commitment creates redundant S3 writes.

6. Assessment Results

During our assessment on the scoped Celestia op-alt-da files, we discovered four findings, all of which were low impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.